Spanning tree construction

Abbiamo visto come la Msg(Flooding) = $\Omega(m)$

La strategia per aggirare il lowerbound sulla msg complexity e' quella di:

i) Costruire uno spanning tree di G → Sottografo connesso di G con n-1 archi

ii) Far girare il protocollo solo sul sottografo G

Introduzione informale

In un sistema distribuito, costruire uno spanning tree T di G vuol dire che bisogna creare una configurazione dove:

a) Ogni entita' x ha un sottoinsieme Tree-Neighbour(x) $\subseteq N(x)$

b) L'insieme di tutti gli archi corrispondenti a Tree-Neighbour(x) $\forall x \in E$ appartengono allo Spanning Tree $\hat{T}$

Da notare che un'entita' $x \in E$ non ha bisogno di conoscere tutto T ma solo quali dei suoi vicini facciamo parte di T

Partendo da questa assermazione ed anticipando che ci troviamo sotto le restrizioni [R] → Connectivity, Total reliability, Bidirectional links, Unique Initiator

Vogliamo un protocollo che permetta ai nodi di conoscere quali dei propri vicini lo siano anche in $\hat{T}$

La soluzione e' tutta dietro la strategia "ask" ovvero:

1) L'initiator $s \in E$ mandera' un messaggio Q? a tutti i suoi vicini

2) Un'entita' $x \neq s$ rispondera' "Yes" alla prima volta che riceve Q? e lo inviera' a tutti gli altri vicini
   Rispondera' "No" ad ogni altra richiesta Q?

3) Ogni entita' termina quando riceve una risposta da tutti i suoi vicini

Praticamente quello che stiamo cercando di progettare è della forma Flood+Reply

Assunzioni e restrizioni del sistema

Come già accennato le assunzioni sono:

i) Connectivity

ii) Total reliability

iii) Unique Initiator

iv) Bidirectional links

Stati possibili

Gli stati possibili sono:

i) Init:

Stato in cui si trova l'unico nodo che fa cominciare la computazione

ii) Idle:

Stato in cui si trovano i nodi prima di ricevere il primo Q?

iii) Active:

Stato in cui si trovano i nodi dopo aver inviato il primo "Yes"

iv) Done:

Stato quiescente in cui i nodi hanno ricevuto tutte le risposte dai propri vicini

Per cui $S = \{Init, Idle, Active, Done\}$

Stato iniziale e finale del sistema

Abbiamo un numero di n configurazioni iniziali possibili

$$S_{Init} = \{Init, Idle\}$$

Ed una sola configurazione finale

$$S_{Term} = \{Done\}$$

Protocollo

Shout:

- Status:

$S = \{Init, Idle, Active, Done\}$

$S_{Init} = \{Init, Idle\}$

$S_{Term} = \{Done\}$

- Restriction: FL, UI

Init:

Spontaneously

root = True

Tree-Neighbour

Send(a?) to N(x)

counter = 0

become Active

Idle

Receiving (a?)

root = False

parent = Sender

Tree-Neighbour = {Sender}

Send(Yes) to Sender

counter = 1

if counter = |N(x)|

become Done

else

Send(a?) to N(x)\{Sender}

become Active

Active

Receiving (Q?)

Send (No) to Sender

Receiving (Yes)

Tree-Neighbours = Tree-Neighbours ∪ {Sender}

counter = counter + 1

if counter = |N(x)|

become Done

Receiving (No)

counter = counter + 1

if counter = |N(x)|

become Done

Terminazione del protocollo

Il protocollo Shout ci assicura che

$\forall x \in V$, x riceverà Q? e risponderà Yes/No ad ogni Q?

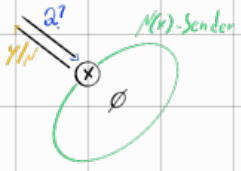
Dim

Visto che il protocollo Shout si basa sul flooding di Q?

Grazie alla dimostrazione del Flood possiamo dire che ogni nodo riceverà Q?

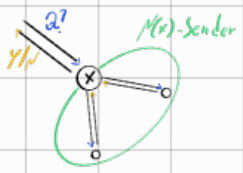
Capiamo ora come i nodi rispondono ai msg Q?

i) Se $N(x) = \{Sender\}$ il caso è banale, risponde subito Yes/No



ii) Se $|N(x)|>1$ nel momento in cui x riceve Q? e risponde Yes

Invierà ad ogni nodo in $N(x)$-Sender il msg Q? e per costruzione di Shout ogni nodo $\in N(x)$-Sender risponderà Yes/No e per l'assunzione di TotalReliability $\times$ $\exists T < \infty$ t.c. x ha ricevuto tutti i messaggi di risposta



Per cui l'algoritmo termina sicuramente ✓

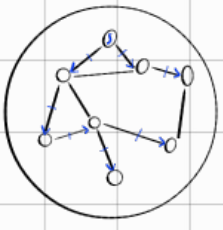
Correttezza del protocollo

Per dimostrare la correttezza del protocollo bisogna mostrare che il sottografo $G'=(V, \bigcup_{x \in V}[Tree\text{-}Neighbour(x)])$ sia uno Spanning Tree di G

Possiamo dimostrare che

1) $x \in Tree\text{-}Neighbours(y) \iff y \in Tree\text{-}Neighbours(x)$
2) Ogni $x \in E$ che invia uno "Yes" è connesso ad s tramite un cammino che termina con l'arco su cui invia "Yes"
3) G' è uno spanning
4) G' non contiene cicli

Grazie alle quali possiamo dire che G' sia uno spanning Tree di G



Dim

1) Vale per costruzione

2) Assumiamo che $\exists x \in E$ che invii "Yes" ma non sia connesso ad s

Iniziamo con l'asserire che x per mandare "Yes" ad un qualunque nodo $y \in E$, deve aver ricevuto un Q? da y

Applichiamo lo stesso ragionamento ad y e, ricorsivamente, diciamo che $\exists$ un nodo $z \in N(s)$ che riceve il primo Q? da s

Ma non gli risponde con "Yes" Assurdo

3) Grazie alla proprietà dimostrata all'interno del broadcast, tutti i nodi $x \in E$ riceveranno almeno un Q? e risponderanno con Yes

Per ② $\forall x \in E$ è connesso quindi ad s

Partendo da qui è facile far vedere come $\forall u,v \in E \; \exists P(u,v)$

Facciamo vedere come, essendo u,v connessi ad s all'interno di G' tramite archi con "Yes"

basta partire da u e percorrere gli archi con "Yes" fino ad s e poi partendo da s arrivo a w

4) Dimostriamo ora che G' non abbia ciclo

Supponiamo che in G' $\exists$ un ciclo diretto $\langle x_0,\dots,x_{k-1}\rangle$ t.c. x_i è il padre di x_{i-1} → Questo vale anche per x_i padre di x_0

Poiché s non invia alcun Yes non può far parte del ciclo

Per ② $\exists$ un cammino $P(s,x) \; \forall x \in E$

Per cui $\exists z \in E$ t.c. $z \notin \langle x_0,\dots,x_k\rangle, P(s,z), (z,x_i) \in C$ → essendo ogni nodo connesso ad s e visto che s∉ ciclo Allora deve esistere un nodo che fa da ponte fra s ed x_i

Ma questo è assurdo perché vorrebbe dire che x_i ha mandato a "Yes"

Analisi della msg complexity

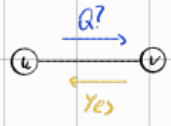
Analizziamo ora quanti messaggi passano su ogni arco

Prendiamo come campione l'arco $(u,v) \in C$

- L'arco $(u,v) \in \tilde{T}$:

  Vorrà dire che v verrà svegliato dal Q? inviato da u, al quale risponderà con 'Yes'

  Non verranno più inviati messaggi su (u,v) per cui il # di msg che passano al massimo su (u,v) sono 2



- L'arco $(u,v) \notin \tilde{T}$:

  Vuol dire che v e u sono stati già svegliati in precedenza da altri msg Q? per cui sia u che v invieranno Q? sull'arco (u,v) a cui entrambi risponderanno con "No"

  Per cui su (u,v) passano al massimo 4 msg



Quindi sapendo che il # di archi in $\tilde{T}$ è esattamente n-1

Abbiamo che

$$Msg(*) = (m-(n-1))\,4 + (n-1)\,2 = 4m - 4n + 4 + 2n - 2 = 4m - 2n + 2$$

Abbassare la msg complexity di Shout

Possiamo notare che la msg complexity possa essere abbassata ancora di più

Per farlo bisogna ragionare sull'utilità dei messaggi "No"

Effettivamente l'invio del msg "No" è ridondante in quanto questo verrà inviato solo se viene ricevuto "Q?" su un arco su cui "Q?" è già stato inviato

A questo punto risulta più efficiente utilizzare come risposta implicita ad un "Q?" il ricevimento di un'altro "Q?"

Per cui utilizzando questo ragionamento avremo su ogni arco sia in T che non solamente 2 msg scambiati

Quindi

$$Msg(Shout+) \leq 2m$$

Risultato di impossibilità di SPT con più sorgenti

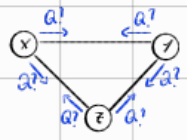
Cosa accade a Shout se eliminassimo l'assunzione di Unique Initiator?

Analizziamo ora il seguente caso dove:

- $G=(V,U)$ è una clique composta da 3 nodi x,y,z
- L'insieme delle sorgenti W di sorgenti è $W=V$ svegliate contemporaneamente

Eseguendo ora il protocollo Shout riscontriamo un problema

su quale nodo non far passare 'Yes' in modo da non creare un ciclo?



Questo non è possibile capirlo in quanto ci troviamo all'interno di una assoluta simmetria

Per cui non è possibile risolvere la task di SPT utilizzando un protocollo deterministico